TraceMSR: A Tool for Synthesizing Sufficient Reasons for Reactive Systems

Nadav Rutman Moshe and Hadar Frenkel

Department of Computer Science, Bar-Ilan University, Ramat Gan, Israel
{nadav.rutman,hadar.frenkel}@biu.ac.il

Abstract. We introduce the tool TraceMSR, the first fully automated tool for the synthesis of sufficient reasons for LTL effects over infinite words. Given an LTL formula φ, which (some of the) traces of a reactive system satisfy, and an example of such a trace π, a sufficient reason provides a minimal explanation for the fact that π satisfies φ. Our tool is able to produce non-temporal reasons, as (possibly infinite) sets of events on the trace, or temporal reasons, in the form of Büchi automata. The two notions give the user a trace-local and a system-global explanation of the same effect. To synthesize non-temporal reasons, TraceMSR uses a reduction to HyperLTL model-checking, while the synthesis of temporal reasons is done via an automata construction, using Spot. Our tool is flexible and allows the user to choose between the different configurations. We evaluate TraceMSR on 987 instances derived from 197 SYNTCOMP systems. TraceMSR computes the minimal temporal sufficient reason for 915 of the instances, and a minimal sufficient reason, or a proof that none exists, for 717 of them, within a timeout of 120 seconds (300 seconds for systems with four inputs).

1 Introduction

In recent years, the questions of causality and explainability have gained great interest from the computer science community, and, in particular, from the verification community [23,2,8]. One line of works studied *actual causality* following the definitions of Halpern and Pearl [18]: Given an LTL specification φ, and a trace π that violates it (that is, $\pi \vDash \neg\varphi$), it aims at finding the actual cause for the facts that π violates φ in the given system [11,12,14]. Tools that implement these definitions include [11], CATS [7] and CORP [14].

A more recent work [16] studied a different notion of explainability, originated from the machine learning community [4] , namely *sufficient reasons*, and provided definitions of sufficient reasons in the setting of reactive systems, and algorithm for synthesizing such reasons. Generally, a sufficient reason is a set of events that is sufficient to observe the same outcome. A *minimal* sufficient reason, is a minimal set of such events. In [16], the authors have studied two notions of sufficient reasons, given a reactive system, an LTL effect, and an infinite trace: The first notion is in the form of a set of events that appear on the given trace, similar to the definition above. The authors manage to find reasons

as infinite sets of events, using a finite representation as lasso traces. The second notion is *temporal* sufficient reasons, that provide a more general explanation, in the form of an automaton corresponding to a temporal property.

While the authors of [16] provided definitions and algorithms for both notions, their implementation was not automatic: each system, trace and effect had to be encoded by hand as a HyperLTL formula for the model checker. This motivated us to present TraceMSR - a fully automated implementation of the definition for sufficient reasons. TraceMSR performs this encoding automatically, and extends the implementation of [16]: (1) each trace quantifier ranges over its own system, which reduces the running time (Section 4.4); (2) the user can choose between two model checkers, AutoHyper and HyperQB; and (3) all the user has to provide is a single input file with the system (as an HOA automaton or an LTS), the effect (as an LTL formula) and, optionally, a trace; the user then chooses the operation from a menu (e.g. computing the MSR or the $MTSR$), and TraceMSR displays the reason it finds, optionally also as a trace (Section 4.5).

Actual causes vs. sufficient reasons. Actual causality in reactive systems [11,12,14,7] provides the most refined cause for a given effect, and the process of finding it is costly and elaborated. (Minimal) sufficient reasons [16] provide any (minimal) set of events that would lead to the same outcome. The two approaches provide different insights: Actual causality finds the specific cause on a specific trace. Sufficient reasons use this specific trace, but are more general, and can be used to learn things both about this specific trace, and about the whole system. Consider, for example, the simple system of Figure 1. The system loops in q_0 until there is a request, upon which the system immediately outputs a grant. After the first request, the system loops in q_1 and does not output any more grants. Consider now the input trace $\{r\}^\omega$, which yields the full trace $\pi = \{r, g\}\{r\}^\omega$, and the LTL effect $\varphi =$ *eventually grant*. The actual cause for φ is $\{(0, r)\}$ - on this trace, only the first input causes an eventual grant. However, a minimal sufficient reason is any $\{(n, r)\}$ for $n \in \mathbb{N}$, i.e., it is sufficient to observe a request at any timepoint to eventually observe a grant. This is since any trace that agrees with π on some request, would have a grant at some point (upon the first request).

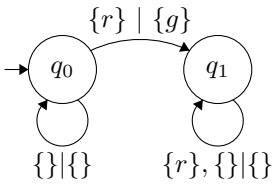


Fig. 1.

If one is interested to know what is the cause for the grant on this particular trace, they should use actual causality. If one is interested in how to observe grants also on other system traces, or, alternatively, how to avoid grants, they should use sufficient reasons. In particular, avoiding the request only on the first timepoint would not avoid the effect of *eventually grant*. Therefore, sufficient reasons provide a much better approach for repair purposes.

Contributions.

- TraceMSR is the first tool to automatically synthesize minimal temporal and non-temporal sufficient reasons for reactive systems (Section 4).

– We characterize the cases in which no minimal sufficient reason exists (Section 3). TraceMSR uses this characterization to give the user more information in these cases, and outputs some sufficient reason instead of a minimal one.
– We evaluate TraceMSR on 987 instances derived from 197 SYNTCOMP systems , with a comparison to three tools for actual causality (Section 5).

The source code of TraceMSR, the benchmarks of Section 5 and the scripts that reproduce its tables and figures are available at `https://github.com/NadavRutmanMoshe/MSR_Finder`. An archived version will be made available on Zenodo by the artifact evaluation deadline.

1.1 Related Work

Sufficient reasons. Sufficient reasons were defined in the context of machine learning, where they explain the decision of a classifier on a given input [4,21,13]. Recently, [16] lifted these definitions to reactive systems and LTL effects, and provided algorithms and complexity results for their verification and synthesis, together with a prototype implementation in which each instance was encoded manually. TraceMSR is the first tool that synthesizes minimal and temporal sufficient reasons for reactive systems automatically.

Causality for reactive systems. Many works study explanations for temporal properties [2,23,15,5], but none of them considers sufficient reasons on infinite traces. The closest line of work lifts the actual causality of Halpern and Pearl [18] to reactive systems, with definitions of non-temporal [11] and temporal [12] actual causes, and a complexity analysis [9]. Three tools implement these definitions: [11] computes actual causes for counterexamples of HyperLTL properties, CATS [7] checks whether a given temporal property is a cause of an effect on a trace, and searches for causes by completing a cause sketch, and CORP [14] synthesizes temporal causes. These tools compute actual causes, while TraceMSR computes sufficient reasons; we compare TraceMSR with the three tools in Section 5. Error explanation for counterexamples [3,17] considers finite traces, while TraceMSR explains infinite traces with general LTL effects.

HyperLTL model checking. TraceMSR reduces the synthesis of minimal sufficeint reasons to HyperLTL model checking (Section 4.3), and uses two model checkers: AutoHyper [6], an explicit-state model checker, and HyperQB [20,19], a bounded model checker that checks it up to a given bound.

2 Preliminaries

2.1 Traces, Temporal Logic and Automata

Given a finite set X, X^* is the set of all finite sequences of elements of X, X^+ the set of all non-empty finite sequences, and X^ω the set of all infinite sequences. For an infinite sequence α, $\alpha[t]$ is its element at position $t \in \mathbb{N}$, and $\alpha[t, \infty)$ is its suffix starting at position t.

Systems and traces. Let AP be a finite set of atomic propositions, where $\mathrm{AP} = \mathrm{IP} \uplus \mathrm{OP}$ with IP the set of input propositions, and OP the set of output propositions. A *reactive system* is $T = (Q, q_0, \delta, \mathrm{IP}, \mathrm{OP})$ with $\delta : Q \times 2^{\mathrm{IP}} \to Q \times 2^{\mathrm{OP}}$; note that δ is a function, i.e. T is *input deterministic*.[1] Here, Q is a finite set of states, $q_0 \in Q$ is the initial state, and δ is the transition function, mapping the current state and the current inputs to the next state and the current outputs.

A *path* of T is an infinite sequence of states $q_0 q_1 q_2 \cdots \in Q^\omega$ that starts in the initial state, together with inputs $i_t \in 2^{\mathrm{IP}}$ and outputs $o_t \in 2^{\mathrm{OP}}$ such that $\delta(q_t, i_t) = (q_{t+1}, o_t)$ for all $t \in \mathbb{N}$. The *trace* induced by the path is $\pi = (i_0 \cup o_0)(i_1 \cup o_1)(i_2 \cup o_2) \cdots \in (2^{\mathrm{AP}})^\omega$, and $\mathrm{TR}(T)$ is the set of all traces induced by paths of T. Since δ is a function, every sequence of inputs induces exactly one trace of T. The *projection* of $\pi \in (2^{\mathrm{AP}})^\omega$ onto a set $A \subseteq \mathrm{AP}$ is $\pi_{\downarrow_A} = (\pi[0] \cap A)(\pi[1] \cap A)(\pi[2] \cap A) \cdots \in (2^A)^\omega$; particularly, $\pi_{\downarrow_{\mathrm{IP}}}$ is the sequence of inputs of π.A trace π is a *lasso* (or *ultimately periodic*) if there are $u \in (2^{\mathrm{AP}})^*$ and $v \in (2^{\mathrm{AP}})^+$ such that $\pi = u \cdot v^\omega$. A lasso is thus a finite representation of an infinite trace. The traces that TraceMSR takes as inputs and returns as outputs are all lasso traces. For two traces π and τ, we write $\tau[t] =_{\{a\}} \pi[t]$ when τ and π agree on the proposition a at time t.

Büchi automata. A nondeterministic Büchi automaton (NBA) is a tuple $\mathcal{A} = (\Sigma, Q, q_0, \Delta, F)$, consisting of a finite alphabet Σ, a finite set of states Q, an initial state q_0, a transition relation $\Delta \subseteq Q \times \Sigma \times Q$, and a set of accepting states $F \subseteq Q$. A *run* of $\mathcal{A}$ is an infinite sequence of states, which starts at q_0, that respects the transition relation. An infinite word α is accepted by $\mathcal{A}$ if it has a run of $\mathcal{A}$ on α that visits F infinitely often, and $\mathcal{L}(\mathcal{A})$ is the set of all accepted words. A system T can be viewed as an NBA over 2^{AP} with $F = Q$ and a transition $(q, i \cup o, q') \in \Delta$ whenever $\delta(q, i) = (q', o)$, so that $\mathcal{L}(T) = \mathrm{TR}(T)$; we use this view when we intersect a system with an automaton.

LTL and HyperLTL. We use Linear Temporal Logic (LTL) [24] and its extension HyperLTL [10], whose formulas are given by the following grammar, where LTL formulas are only of type ψ, i.e., without trace quantification.

$$\psi ::= a \mid \neg\psi \mid \psi \vee \psi \mid \bigcirc\psi \mid \psi \,\mathcal{U}\, \psi \qquad\qquad \Phi ::= \exists \pi.\, \Phi \mid \forall \pi.\, \Phi \mid \psi$$

where for LTL formulas, $a \in \mathrm{AP}$; and for HyperLTL formulas $a = p_\pi$, for $p \in \mathrm{AP}$ and a trace variable π. That is, a HyperLTL formula is an LTL formula in which every proposition is indexed by a trace variable, preceded by trace quantifiers. $\bigcirc$ (*next*) and $\mathcal{U}$ (*until*) are temporal operators. We use the derived operators $\Diamond\psi = \mathit{true}\,\mathcal{U}\,\psi$ (*eventually*) and $\Box\psi = \neg\Diamond\neg\psi$ (*globally*), and the usual Boolean connectives.

LTL semantics. We define when an infinite word $\alpha \in (2^{\mathrm{AP}})^\omega$ satisfies an LTL formula ψ (denoted as $\alpha \vDash \psi$) as follows:

$$\alpha \vDash a \qquad\qquad \text{iff} \quad a \in \alpha[0]$$

[1] This indeed holds for all of the SYNTCOMP generated benchmarks, derived from [7].

$$
\begin{array}{lll}
\alpha \vDash \neg\psi & \text{iff} & \alpha \nvDash \psi \\
\alpha \vDash \psi_1 \vee \psi_2 & \text{iff} & \alpha \vDash \psi_1 \text{ or } \alpha \vDash \psi_2 \\
\alpha \vDash \bigcirc \psi & \text{iff} & \alpha[1,\infty) \vDash \psi \\
\alpha \vDash \psi_1 \,\mathcal{U}\, \psi_2 & \text{iff} & \exists t \in \mathbb{N}.\ \alpha[t,\infty) \vDash \psi_2 \text{ and } \forall t' < t.\ \alpha[t',\infty) \vDash \psi_1
\end{array}
$$

The language of ψ is $\mathcal{L}(\psi) = \{\alpha \in (2^{\mathrm{AP}})^\omega \mid \alpha \vDash \psi\}$, and a system T satisfies ψ, denoted $T \vDash \psi$, if $\mathrm{TR}(T) \subseteq \mathcal{L}(\psi)$. Every LTL formula ψ can be translated into an NBA $\mathcal{A}_\psi$ with $\mathcal{L}(\mathcal{A}_\psi) = \mathcal{L}(\psi)$ [25].

A HyperLTL formula is evaluated over the traces of the system: $\exists\pi$ requires some trace, and $\forall\pi$ every trace, to satisfy the rest of the formula. We provide the full HyperLTL semantics in Section A. For example, $\exists\pi.\,\exists\pi'.\,\Diamond g_\pi \wedge \Box\neg g_{\pi'}$ states that the system has a trace on which g eventually holds, and another trace on which g never holds.

2.2 Sufficient Reasons

In the following, we fix a system T, an LTL effect φ over AP, and a trace $\pi \in \mathrm{TR}(T)$ with $\pi \vDash \varphi$; we abbreviate this triple as $\langle\varphi, T, \pi\rangle$. We define reasons as sets of events on π, i.e. subsets of $\mathbb{N} \times \mathrm{IP}$.

Definition 1 (Sufficient Reasons for LTL Effects [16]). *Let $R \subseteq \mathbb{N} \times \mathrm{IP}$.*

- *R is a* sufficient reason *(SR) for $\langle\varphi, T, \pi\rangle$ if for every $\tau \in TR(T)$ such that $\tau[t] =_{\{a\}} \pi[t]$ for all $\langle t, a\rangle \in R$, we have $\tau \vDash \varphi$.*
- *R is a* minimal sufficient reason *(MSR) for $\langle\varphi, T, \pi\rangle$ if R is a SR for $\langle\varphi, T, \pi\rangle$ and there is no SR R' with $R' \subsetneq R$.*

We denote by $SR_{\varphi,T,\pi}$ and $MSR_{\varphi,T,\pi}$ the sets of **all** SRs and of **all** MSRs for $\langle\varphi, T, \pi\rangle$, respectively. Let $\mathit{Reasons} = \mathcal{P}(\mathbb{N} \times \mathrm{IP})$ (we think of it as the set of all candidate reasons). Then, we have $MSR_{\varphi,T,\pi} \subseteq SR_{\varphi,T,\pi} \subseteq \mathit{Reasons}$.

Since T is input deterministic, the only trace agreeing with π on *all* inputs at *all* time points is π itself, which satisfies φ; hence:

Observation 1 ([16]) *$R_{all} = \mathbb{N}\times\mathrm{IP}$ is a SR for $\langle\varphi, T, \pi\rangle$. In particular $SR_{\varphi,T,\pi} \neq \emptyset$.*

Temporal sufficient reasons. Instead of a set of events on π, a temporal sufficient reason is a trace property over the inputs, i.e. a set $P \subseteq (2^{\mathrm{IP}})^\omega$. A trace τ satisfies P if its sequence of inputs $\tau_{\downarrow \mathrm{IP}}$ is in P.

Definition 2 (Temporal sufficient reasons [16]). *Let P_R be a trace property over* IP*.*

- *P_R is a* temporal sufficient reason *(TSR) for $\langle\varphi, T, \pi\rangle$ if π satisfies P_R, and every $\tau \in TR(T)$ that satisfies P_R also satisfies φ.*
- *P_R is a* minimal temporal sufficient reason *(MTSR) if there is no TSR $P_{R'}$ with $P_{R'} \supsetneq P_R$.*

For example, consider the biased arbiter of [16] (Figure 3), with IP $= \{r_1, r_2\}$ and OP $= \{g_1, g_2\}$, in which every request r_1 is granted by g_1 in the same step, and g_1 is never given without r_1. For $\pi = \{r_1, g_1\}^\omega$ and $\varphi = \square\diamondsuit g_1$, both $\mathcal{L}(\square r_1)$ and $\mathcal{L}(\square\diamondsuit r_1)$ are TSRs, and $\mathcal{L}(\square\diamondsuit r_1)$ is the MTSR [16]. Note the direction: a TSR is a constraint on the inputs, and the less it constrains, the larger it is; here $\mathcal{L}(\square\diamondsuit r_1) \supsetneq \mathcal{L}(\square r_1)$. Hence, minimality here means maximality with respect to $\supseteq$, i.e. the weakest constraint that still guarantees the effect. The MTSR always exists, is unique, and is ω-regular:

Theorem 1 ([16, Theorem 5]). *The MTSR for $\langle \varphi, T, \pi \rangle$ is $\mathcal{L}\big((T \cap \mathcal{A}_\varphi)_{\downarrow \text{IP}}\big)$, that is, the projection onto* IP *of the product of T with an automaton $\mathcal{A}_\varphi$ for φ.*

Theorem 1 gives a direct construction of the $MTSR$, which TraceMSR implements (Section 4). Since this construction only uses operations on automata, and no HyperLTL model checking, computing the $MTSR$ is much cheaper than computing an MSR (Section 5).

3 Existence of Minimal Sufficient Reasons

In the following section, we present two results which we later use for synthesizing MSRs in TraceMSR. This section presents new results for the case in which there is no MSR. An example for this case was presented in [16] with the effect $\varphi = \square\diamondsuit g_1$ on the trace $\pi = \{r_1, g_1\}^\omega$, where $\mathbb{N} \times \{r_1\} \supsetneq \mathbb{N}_{>1} \times \{r_1\} \supsetneq \mathbb{N}_{>2} \times \{r_1\} \supsetneq \cdots$ are all sufficient ([16], Example 2). In the following, we generalize what happens in such cases.

Theorem 2 characterizes the cases in which there is no MSR; in these cases, TraceMSR reports this to the user and shows a sufficient (but not minimal) reason instead. Proposition 1 shows that a reason found by a bounded search must be verified, which is why TraceMSR verifies every reason that HyperQB finds (Section 4.2).

Throughout this section, we fix a system T, an LTL effect φ and a trace $\pi \in \text{TR}(T)$ with $\pi \vDash \varphi$, as in Definition 1.

Theorem 2. *If $MSR_{\varphi,T,\pi} = \emptyset$ then $|SR_{\varphi,T,\pi}| \geq \aleph_0$ and every $R \in SR_{\varphi,T,\pi}$ is infinite. Formally,*

$$MSR_{\varphi,T,\pi} = \emptyset \implies \big(|SR_{\varphi,T,\pi}| \geq \aleph_0\big) \wedge \big(\forall\, R \in SR_{\varphi,T,\pi} : |R| \geq \aleph_0\big).$$

We prove the two contrapositives separately: a minimal reason exists as soon as there are *finitely many* sufficient reasons, or as soon as *at least one* sufficient reason is finite.

Lemma 1. $|SR_{\varphi,T,\pi}| < \aleph_0 \implies MSR_{\varphi,T,\pi} \neq \emptyset$.

Proof. Assume $SR_{\varphi,T,\pi}$ is finite. It is non-empty by Observation 1. Minimality is taken with respect to set inclusion, which is a partial order, and every non-empty finite partially ordered set has (at least one) minimal element. Hence $MSR_{\varphi,T,\pi} \neq \emptyset$. □

Lemma 2. $\exists R \in SR_{\varphi,T,\pi} : |R| < \aleph_0 \Longrightarrow MSR_{\varphi,T,\pi} \neq \emptyset$.

Proof. Suppose towards a contradiction that $MSR_{\varphi,T,\pi} = \emptyset$. Then no $R \in SR_{\varphi,T,\pi}$ is minimal, so for every $R \in SR_{\varphi,T,\pi}$ there is some $R' \in SR_{\varphi,T,\pi}$ with $R' \subsetneq R$. Let $R_0 \in SR_{\varphi,T,\pi}$ be finite with $|R_0| = n$ (according to our assumption, such R_0 exists). Applying this repeatedly yields a strictly decreasing chain

$$\cdots \subsetneq R_2 \subsetneq R_1 \subsetneq R_0,$$

and each step removes at least one element. After at most n steps we reach $R_m \in SR_{\varphi,T,\pi}$ with $|R_m| = 0$, i.e. $R_m = \emptyset$. But $\emptyset$ is minimal with respect to inclusion, so $R_m \in MSR_{\varphi,T,\pi}$, contradicting $MSR_{\varphi,T,\pi} = \emptyset$. □

Proof (of Theorem 2). Immediate from the contrapositives of Lemmas 1 and 2: $MSR_{\varphi,T,\pi} = \emptyset$ implies $|SR_{\varphi,T,\pi}| \geq \aleph_0$ and that no $R \in SR_{\varphi,T,\pi}$ is finite. □

When there is no MSR, the same construction as in the proof of Lemma 2 gives an infinite chain:

Corollary 1. *If $MSR_{\varphi,T,\pi} = \emptyset$ then there is an infinite strictly descending chain $R_0 \supsetneq R_1 \supsetneq R_2 \supsetneq \cdots$ of sufficient reasons for $\langle \varphi, T, \pi \rangle$.*

Proof. If $MSR_{\varphi,T,\pi} = \emptyset$ then no $R \in SR_{\varphi,T,\pi}$ is minimal, so every $R \in SR_{\varphi,T,\pi}$ has some $R' \in SR_{\varphi,T,\pi}$ with $R' \subsetneq R$. $SR_{\varphi,T,\pi} \neq \emptyset$ by Observation 1, so starting from $R_0 = R_{\text{all}}$ and repeating this choice produces the required chain. □

We showed that not having an MSR means that there is certainly an infinite strictly decreasing chain of sufficient reasons (Corollary 1). Therefore, when TRACEMSR identifies this case, it shows the user a sufficient reason instead, and reports that an infinite decreasing chain of sufficient reasons exists (Section 4.2).

Restricting the search to bounded reasons is not complete. A common way to make the search tractable is to restrict it to reasons of a bounded shape, as done in [11]. For fixed $P, L \geq 1$, a reason R is *(P, L)-bounded* if $\langle t, a \rangle \in R \Leftrightarrow \langle t{+}L, a \rangle \in R$ for all $t \geq P$, i.e. R has the shape of a lasso with a prefix of size P and a loop of size L. We denote the set of (P, L)-bounded reasons by $\mathit{Reasons}_{P,L}$. In the following, we show that restricting the search to $\mathit{Reasons}_{P,L}$, for P and L given in advance, does not always give the right result. The problem lies in minimality: a reason that is minimal among the finitely many reasons in $\mathit{Reasons}_{P,L}$ is not necessarily minimal. Verifying the reason we found removes such wrong answers, but it does not solve the problem: when no MSR exists, or when every MSR lies outside the bound, the search does not find a minimal reason, and it cannot tell these two cases apart. Consider, for example, IP $= \{a, b\}$, OP $= \{q, r\}$ and a system in which the output q repeats the input a with a delay of one step (q holds at time $t{+}1$ iff a holds at time t), and the output r holds at time 2 iff b holds at time 1, and is false at all other times (We provide a figure of the system in Appendix B). Let $\varphi = \Box\Diamond q \vee \Diamond r$ and $(P, L) = (3, 1)$:

π	$\{\}$	$\{a, b\}$	$\{a, q, r\}$	$(\{a, q\})^\omega$
R_b	$\emptyset$	$\{b\}$	$\emptyset$	$(\ \emptyset\ )^\omega$
R_a	$\emptyset$	$\emptyset$	$\emptyset$	$(\ \{a\}\ )^\omega$

$R_b = \{\langle 1, b\rangle\}$ guarantees r, so it is an MSR. $R_a = \{\langle i, a\rangle \mid i \geq 3\}$ guarantees that q holds infinitely often, so it is sufficient as well, and it is minimal in $\mathit{Reasons}_{3,1}$ (the reasons with a prefix of size 3 and a loop of size 1), since its only proper subset in $\mathit{Reasons}_{3,1}$ is $\emptyset$, which is not sufficient (a and b may be false forever). But R_a is not an MSR in general, since $R_a \setminus \{\langle 5, a\rangle\}$, a subset of R_a, is also sufficient. The same argument works for any bound:

Proposition 1. *Let T be the system of Figure 8, $\varphi = \square\diamondsuit q \vee \diamondsuit r$, and $\pi = \{\}\{a, b\}\{a, q, r\}(\{a, q\})^\omega$. For all P, $L \geq 1$ and $0 \leq j < L$, the reason $R = \{\langle t, a\rangle : t \geq P,\ t \equiv j \bmod L\}$ is minimal among the sufficient reasons in $\mathit{Reasons}_{P,L}$ but is not an MSR. Hence $MSR_{\varphi,T,\pi} \cap \mathit{Reasons}_{P,L} \subsetneq \min(SR_{\varphi,T,\pi} \cap \mathit{Reasons}_{P,L})$ for every bound (P, L).*

Proof (sketch). R is sufficient, since every trace that agrees with π on R has infinitely many a's, hence infinitely many q's, and therefore satisfies φ. R is minimal in $\mathit{Reasons}_{P,L}$, since the only set in $\mathit{Reasons}_{P,L}$ that is contained in R, other than R itself, is $\emptyset$, which is not sufficient. However, R is not an MSR: the reason $R' = \{\langle t, a\rangle : t \geq P + L,\ t \equiv j \bmod L\}$ is also sufficient, and $R' \subsetneq R$. □

We showed that a bounded search may find reasons that are minimal within the search, but are not an MSR. TraceMSR finds MSRs by solving a HyperLTL model checking problem, and allows the user to choose between the model checkers AutoHyper and HyperQB. In case the user chooses HyperQB, which uses a bounded search, we also verify the resulting reason. In Appendix C we show that we can reduce this check to automata operations, therefore limiting its effect on the overall runtime.

4 Tool Overview

In the following, we describe our tool TraceMSR. TraceMSR implements and fully automates the algorithms of [16] for the generation of MSRs and $MTSR$s. For the generation of MSRs, we reduce to HyperLTL model checking, which finds a finite representation of the possible infinite reason as a lasso trace (Section 4.3, and [16, Section 5]). For the generation of $MTSR$s, we use automata operations via spot (Theorem 1).

4.1 Input format

The input for TraceMSR is a single plain-text file with five labeled blocks, plus an optional sixth:

```
[info]     Biased arbiter of ATVA'26 (Fig. 1) ...
[inputs]   "r1" "r2"
[effect]   F "g2"
[lasso]    AP: "r1" "r2" "g1" "g2"
           Prefix: {1 3}
           Loop:   {}
[cause]    !"r1" U ("r2" & !"r1")    (optional; only --check reads it)
[system]   HOA: v1
```

```
Start: 0
AP: 4 "r1" "r2" "g1" "g2"
...
--END--
```

The blocks are the following.
- `[info]`: a description of the instance; TraceMSR ignores it.
- `[inputs]`: declares the set IP of input propositions.
- `[effect]`: the effect φ, the LTL formula for which we want to find a reason.
- `[lasso]`: declares the atomic propositions AP and the trace π, given as a prefix and a loop, This block is optional, since three of the four operations do not need a trace (Section 4.2).
- `[cause]`: optional; a candidate cause, used only with the option `--check` to verify it (Section 4.2).
- `[system]`: the system T, given as an automaton in the HOA format [1] or as a labeled transition system.

Checking input determinism. Our definitions assume that the system is input deterministic (Section 2), that is, every sequence of inputs induces a single sequence of outputs. This guarantees that a sufficient reason always exists, namely $R_{\text{all}} = \mathbb{N} \times \text{IP}$ (Observation 1). Therefore, TraceMSR checks that the system is input deterministic, and rejects it otherwise.

4.2 The Four Operations

TraceMSR has four main operations. Operations 1–3 synthesize an MSR; if no MSR exists (Theorem 2, Section 3), we report this and display a sufficient reason instead (using `--hyperQB`, we get a different display, see below). Operation 4 synthesizes the $MTSR$, displays it as a Büchi automaton over IP, and outputs an image of it. We list the four possible operations below. Given the input file, TraceMSR opens a menu to choose the desired operation (Figure 4).

(1) Synthesizing an MSR for a given trace.
(2) Finding a trace π that satisfies φ (if one exists), and then synthesizing an MSR for this trace.
(3) Model checking φ: finding a trace that does not satisfy φ (if one exists), and then synthesizing an MSR for this trace and the effect $\neg\varphi$.
(4) Synthesizing the $MTSR$ using spot.
  (4a) With the flag `--check`: checking whether a given candidate reason is equal to the $MTSR$.

In operations (1)–(3), the defaults model checker is AutoHyper [6]. `--hyperQB` uses HyperQB [19] instead, and `--with-tau` also shows the trace representation of the reason (Section 4.3). Operations (2)–(4) do not require a predefined trace. Operation (4a), is a reason verification instead of synthesis: the language of the reason given in the `[cause]` block is compared with the $MTSR$, and TraceMSR reports whether the two are equal. When using `--hyperQB`, the MSR is computed with HyperQB 2.0 (in operations (2) and (3), the original trace is still found by AutoHyper). Since HyperQB is a bounded model

checker, TraceMSR verifies every reason that HyperQB finds, using spot. See Section 3 and Appendix C. If HyperQB does not find an MSR, TraceMSR prints a sufficient reason instead, and whether an MSR exists remains unknown.

4.3 Encoding MSRs in HyperLTL

TraceMSR synthesizes an MSR by HyperLTL model checking, using the encoding of [16] (Sec. 5). A reason R is represented by a trace τ over IP, where a_τ holds at time t iff $\langle t, a\rangle \in R$; τ is the *trace representation* of R [16, Def. 1].

We use two abbreviations. The first one states that the traces π and π' agree on the events of the reason τ, i.e., whenever a_τ holds, a has the same value on π and on π':

$$\mathit{agree}_\tau(\pi, \pi') \;=\; \square \bigwedge_{a \in \mathrm{IP}} \big(a_\tau \to (a_\pi \leftrightarrow a_{\pi'})\big).$$

The second one states that the reason τ' is strictly smaller than the reason τ: every event of τ' is also an event of τ (left conjunct), and the two reasons differ in at least one event (right conjunct):

$$\tau' \subsetneq \tau \;=\; \square \bigwedge_{a \in \mathrm{IP}} (a_{\tau'} \to a_\tau) \;\wedge\; \neg\square \bigwedge_{a \in \mathrm{IP}} (a_{\tau'} \leftrightarrow a_\tau).$$

Then τ is an MSR for $\langle \varphi, T, \pi\rangle$ iff (1) every trace that agrees with π on τ satisfies φ, and (2) for every smaller reason $\tau' \subsetneq \tau$, some trace agrees with π on τ' but violates φ (the formulas ψ_{SR} and ψ_{MSR} of [16]). TraceMSR checks the following formula, in which each line is one of these conditions:

$$\begin{aligned}\Phi_{MSR} \;=\; \exists \pi.\, \exists \tau.\, \forall \pi_2.\, \forall \tau'.\, \exists \pi_3.\;\; & \mathit{fix}_\pi \\ & \wedge \big(\mathit{agree}_\tau(\pi, \pi_2) \to \varphi_{\pi_2}\big) && (1)\\ & \wedge \big(\tau' \subsetneq \tau \to (\mathit{agree}_{\tau'}(\pi, \pi_3) \wedge \neg\varphi_{\pi_3})\big) && (2)\end{aligned}$$

Here, fix_π states that the inputs of π are those of the trace π that we explain (given in the `[lasso]` block in operation (1), and found by TraceMSR in operations (2) and (3)). If Φ_{MSR} holds, the witness for τ is the trace representation of an MSR R for $\langle \varphi, T, \pi\rangle$, and TraceMSR reads R off this witness. Unlike [16], where all quantifiers range over one system that combines traces and reasons, TraceMSR uses a separate system for each kind of trace (Section 4.4).

4.4 Architecture

Figure 2 shows the structure of TraceMSR. First, TraceMSR parses the input file and checks that the system is input deterministic (Section 4.1). Then, the computation splits into two parts.

MSR (upper part). TraceMSR first needs a trace π: in operation (1) it is given in the `[lasso]` block, and in operations (2) and (3) it is found by AutoHyper, by checking $\exists \pi.\, \varphi_\pi$ or $\exists \pi.\, \neg\varphi_\pi$.
Next, TraceMSR builds one system for each kind of trace in Φ_{MSR}:

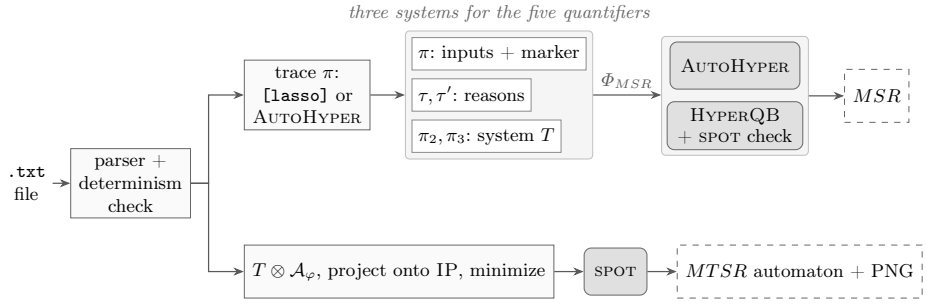


Fig. 2. The structure of TraceMSR. Rounded boxes are external tools, and dashed boxes are the outputs shown to the user. The upper part computes an MSR (Section 4.3, Section 4.4), and the lower part computes the $MTSR$.

– τ and τ' are reasons, so they range over all sequences in $(2^{\text{IP}})^\omega$;
– π ranges over $(2^{\text{IP}})^\omega$ as well, with one additional proposition used by fix_π, since Φ_{MSR} only uses its inputs;
– only π_2 and π_3 range over the system T itself.

In [16], all five quantifiers range over one augmented system, which contains both the traces of T and the traces that represent reasons [16, Def. 2]. TraceMSR instead lets each quantifier range over its own system, a feature of AutoHyper, so that the states of T appear in two copies of the product instead of five; this improved the running time in most of our experiments. Φ_{MSR} is then checked by AutoHyper or by HyperQB, in which case the reason is then verified with spot (Section 4.2). If an MSR is found, TraceMSR shows it to the user as the events of the reason, marked on the trace (see Figure 5a in Section 4.5).

$MTSR$ (lower part). TraceMSR uses spot to compute the product of the system with an automaton for φ, project it onto IP, and minimize it (Theorem 1). The resulting automaton is output in the HOA format and as an image.

4.5 A Session with TraceMSR

We use the system of Figure 3, taken from [16]. The system is biased towards r_1: if r_1 and r_2 are requested at the same time, g_1 is granted and not g_2, and once r_1 is requested, g_2 is never granted again. We use the effect $\varphi = \Diamond g_2$ on the trace $\pi = \{r_2, g_2\}(\{\})^\omega$.

Running TraceMSR on the input file opens the menu of Figure 4. The user selects operation (1) (an MSR for the given trace) and operation (4) (the $MTSR$), and starts the run.

Operation (1) returns the MSR $R = \{\langle 0, r_1\rangle, \langle 0, r_2\rangle\}$ (Figure 5a), meaning that every trace in which r_1 is false and r_2 is true at time 0 satisfies the effect. By minimality, removing one of these events from R results in a reason that is not sufficient. This result makes sense: if r_1 were not forced to be false at time 0, it could be requested at time 0, and then g_2 would never be granted.

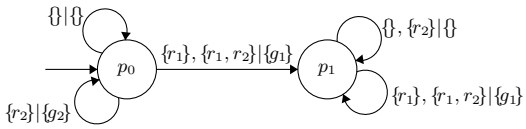


Fig. 3. The biased arbiter of [16, Fig. 1], a system over IP = $\{r_1, r_2\}$ and OP = $\{g_1, g_2\}$. An edge labeled $i|o$ reads the inputs i and produces the outputs o.

```
Select operations for biased_arbiter.txt
   [x] 1. MSR with the given PI
   [ ] 2. MSR for Phi — find a PI automatically
   [ ] 3. Model checking of Phi (counterexample + why)
   [x] 4. MTSR — Minimal Temporal Sufficient Reason
 >     ▶ Start
ENTER/SPACE select · ↑/↓ move · a all · ENTER on Start runs · q quit
```

Fig. 4. The menu of TraceMSR, with the operations 1 and 4 selected.

```
Effect: F "g2"

Status: SAT (Minimal Sufficient Reason Found)

PI:
    {g2,r2} ({})^ω

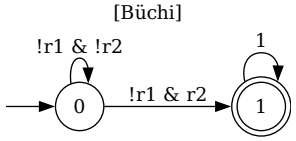
Reason R:
    { (r1,0), (r2,0) }

=============================================
TIME  | ACTUAL TRACE (Pi)   | REASON (R)
=============================================
0     | r1=F, r2=T          | <-- R pins r1, r2
----- + ------------------- + ---------------
CYCLE | (Repeating Pattern) |
----- + ------------------- + ---------------
1     | r1=F, r2=F          |
=============================================

time: 0.7s
```

(a) Operation 1: an MSR

```
found MTSR:  (green = accepting states/edges)
HOA: v1
States: 2
Start: 0
AP: 2 "r1" "r2"
acc-name: Buchi
Acceptance: 1 Inf(0)
--BODY--
State: 0
[!0&!1] 0
[!0&1] 1
State: 1 {0}
[t] 1
--END--
```



(b) Operation 4: the $MTSR$

Fig. 5. The output of TraceMSR on the biased arbiter (Figure 3), with the effect $\varphi = \Diamond g_2$ and the trace $\pi = \{r_2, g_2\}(\emptyset)^\omega$, for the operations 1 and 4.

Operation (4) returns the $MTSR$ (Figure 5b), which generalizes the inputs that are needed to satisfy the effect. We get $MTSR = \mathcal{L}(\neg r_1 \mathcal{U}(r_2 \wedge \neg r_1))$, meaning that r_1 cannot be requested until r_2 is requested without r_1, which is expected for our system.

5 Evaluation

In this section, we evaluate TraceMSR, and compare its results with those of existing tools for actual causality in reactive systems. Section 5.1 describes the

		MSR		$MTSR$		
$\|\text{IP}\|$	instances	solved	avg. t (s)	solved	avg. t (s)	avg. $\|A\|$
1	176	97 %	0.86	100 %	0.20	3.1
2	348	82 %	5.35	94 %	0.26	5.8
3	299	70 %	19.60	90 %	1.46	5.2
4*	164	29 %	118.74	84 %	1.95	6.8
all*	987	72 %	16.05	92 %	0.86	5.3

Table 1. Running TraceMSR on the 987 instances. For MSR, the time is that of the faster model checker; $|A|$ is the number of states of the $MTSR$ automaton. Averages are over the solved instances. *For $|\text{IP}| = 4$, the timeout was 300 s, and 120 s otherwise.

benchmarks and setup, Section 5.2 describes the evaluation of TraceMSR, and Section 5.3 compares TraceMSR with each of the other tools.

5.1 Benchmarks and setup

We use the benchmarks presented in CATS [7], since there is no standard set of benchmarks for explanations in reactive systems. CATS uses SYNTCOMP [22] specifications together with the systems that `ltlsynt` synthesized for them, restricted to systems with at most 300 states ($|Q| \leq 300$), limiting the input propositions to $1 \leq |\text{IP}| \leq 4$, and at least one output proposition. We take the same 204 systems as they are. These systems are input deterministic (Section 2), except that `ltlsynt` sometimes leaves an output unconstrained (i.e. for a given input, it does not specify which value the output takes). In this case, we fix the output to a concrete value; the system still satisfies its specification.

As in CATS, we use random LTL effects generated by `randltl` (tree size 22–30), and random traces. In addition, we keep an instance only if (1) the trace satisfies the effect, and (2) the effect is not trivial, i.e. it is not satisfied by all traces of the system (otherwise the MSR is the empty set). With these filters, we obtain 987 instances over 197 of the 204 systems.

We ran all experiments under Windows Subsystem for Linux, on an Intel Core i7-8565U (1.80 GHz) with 16 GB of RAM. We used a timeout of 120 s for each tool on each instance with $|\text{IP}| \leq 3$, and of 300 s for $|\text{IP}| = 4$. The instances were run one by one: all tools computed their result on an instance before we moved on to the next one. When computing an MSR, we consider an instance solved if TraceMSR returns an MSR, or proves that no MSR exists (Theorem 2).

5.2 Evaluation of TraceMSR

Table 1 shows that, within the timeout, TraceMSR solves 669 of the 823 instances with $|\text{IP}| \leq 3$ (120 s), and 48 of the 164 instances with $|\text{IP}| = 4$ (300 s; 23 of them within 120 s), i.e. 717 of the 987 instances. The rows with $|\text{IP}| = 4$ are marked with * in Table 1, since their timeout differs. The running time correlates

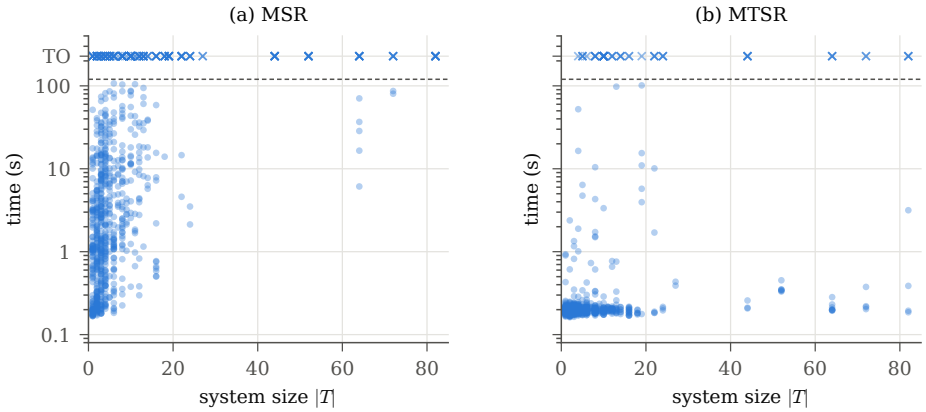


Fig. 6. Running time w.r.t. system size (number of states) on the 823 instances with $|\text{IP}| \leq 3$. A cross is an instance without an answer: (a) no MSR found within the timeout, (b) no $MTSR$ within the timeout or the memory limit.

		solved			avg. t (s), solved by both	
$\lvert\text{IP}\rvert$	instances	AutoHyper	HyperQB	both	AutoHyper	HyperQB
1	176	171	117	116	1.64	0.49
2	348	280	160	154	4.39	7.86
3	299	209	80	78	10.69	23.43
4*	164	43	24	19	106.37	75.37
all*	987	703	381	367	10.14	12.34

Table 2. Computing MSRs with AutoHyper vs. HyperQB over the instances of Table 1: number of instances that each model checker solves, and average running time on the instances that both solve. *For $|\text{IP}| = 4$, timeout is 300 s, and 120 s otherwise).

well with the number of inputs of the system: as described in Section 4.4, three of the five quantifiers of Φ_{MSR} range over all input sequences $(2^{\text{IP}})^\omega$, whose alphabet is exponential in $|\text{IP}|$. Computing the $MTSR$ is much faster: instead of solving a HyperLTL model checking problem, it only uses automata operations of spot. Figure 6 shows that the running time of MSR does not correlate well with the size of the system, and that $MTSR$ takes around 0.2 s in most cases. Table 2 compares between the different model checkers, AutoHyper and HyperQB, on the task of MSR generation. The run-times of HyperQB include also the validity check of the reason (Section 3, Appendix C). We note that AutoHyper manages to solve more instances, but, on those that are solved by both model checkers, HyperQB performs better in some cases (particularly, with $|ip| = 1, 4$).The run-times we report for other tables and figures are of the best configuration. Figure 10 in appendix D provides a visual analysis of this comparison.

		avg. t (s)		avg. size	
|IP|	instances	tool in [11]	MSR	$\|C\|$	$\|R\|$
1	45	0.37	0.93	2.4	3.7
2	58	0.84	2.10	2.4	4.4
3	43	0.56	23.80	2.2	5.0
4*	6	1.08	204.83	2.2	2.8
all*	152	0.63	15.89	2.3	4.3

Table 3. MSR against [11], on instances where both return an explanation (excluding 16 instances on which no MSR exists). $|R| = |u|+|v|$ of the smallest uv^ω representing R. We take the smallest minimal cause returned by [11]. * T/O of 300 s instead of 120 s.

5.3 Comparison with Actual Causality Tools

The tools below compute causes using definitions that differ from ours, as discussed in Section 1. We therefore compare them where we can: by running time, and by the size of the answers where the two are comparable. The motivation to provide these comparisons is two-fold: first, and mainly, to demonstrate how the different causality definitions affect the runtime and the sizes of the produced reasons. This can help the user choose better between the different notions. Second, these are the most recent tools studying explainability of reactive systems and provide explanations that are similar in nature to those we provide. Averages are over the instances both tools solved. Table 7 summarizes all comparisons.

5.3.1 MSR vs. [11]. [11] computes actual causes for counterexamples of HyperLTL formulas. Both an actual cause and an MSR are sets of input events on the same trace: a cause is a minimal set whose change, possibly together with a contingency, falsifies φ [11], while an MSR is a minimal set that guarantees φ. However, we note two important differences between the tools: First, [11] does not provide an infinite set of events as a cause, which we do. In particular, as we showed in Section 3, limiting sufficient reasons to a bounded structure (as done in [11]) is not always possible. Second, [11] provide explanations to *hyperproperty* effects, while we only handle LTL effects. All comparisons here limit the HyperLTL formulas for [11] to be LTL formulas.

Table 3 shows that [11] is faster and returns shorter explanations. This is expected, since it only searches for finite causes, using a SAT solver, while we have to solve a HyperLTL model-checking instance to find lasso traces that represent the reason. See also Figure 9 in Section D.

5.3.2 $MTSR$ vs. CATS [7] CATS is a tool for the verification and sketching of temporal actual causes, i.e., causes in the form of an LTL effect. The main purpose of CATS is cause verification. That is, given an LTL formula as a cause candidate, verify whether this is an actual cause. However, it can also sketch causes using a bounded search. We discuss these two in the following.

	CATS		*MTSR*		
instance	actual cause	t (s)	*MTSR*	t (s)	$\lvert A\rvert$
Spurious Arbiter	×	1.93	×	0.32	1
Arbiter simple	✓	1.81	✓	0.30	3
Arbiter	✓	3.41	×	0.33	3
Example 1	✓	1.73	✓	0.30	4
Example 1, mod	×	1.64	✓	0.31	4
Example 6, odd	✓	2.74	×	0.31	2
Example 6, globally	×	1.19	×	0.30	2
Example 8	✓	1.68	✓	0.38	3
Example 8, mod	×	1.55	✓	0.34	3
TP Left	×	0.86	✓	0.31	1
TP Right	✓	1.61	✓	0.31	2

Table 4. The hand-crafted instances of Table 1 of [7], each with its candidate cause φ_C. 'actual cause': the answer of CATS; '*MTSR*': the answer of `--check` (Section 4.2), i.e. whether φ_C is the *MTSR*. $|A|$ is the number of states of the *MTSR* automaton.

5.3.2.1 *MTSR* vs. CATS, Verification CATS gets a candidate cause φ_C and decides whether it is an *actual* cause. For the same φ_C, we use the check of TraceMSR (`--check`, Section 4.2), which decides whether φ_C is the *MTSR*, i.e. whether $\mathcal{L}(\varphi_C) = \mathcal{L}(MTSR)$ provided with the given system and trace. Table 4 provides a comparison against the 11 hand-crafted instances of Table 1 of [7]. Figure 7 provides a comparison over randomly-generated candidates: As in [7], we used `randltl` to generate a candidate cause φ_C for each instance with $|\text{IP}| \leq 3$. On 325 of them, `randltl` simplified the formula to one without inputs; we removed those, and were left with 498 candidates. On 30 of them, one of the tools timed out or failed; they are shown as crosses in Figure 7.

MTSR is faster on 446 of the 468 candidates both tools answered, with average runtime of 0.62 s against 1.00 s.

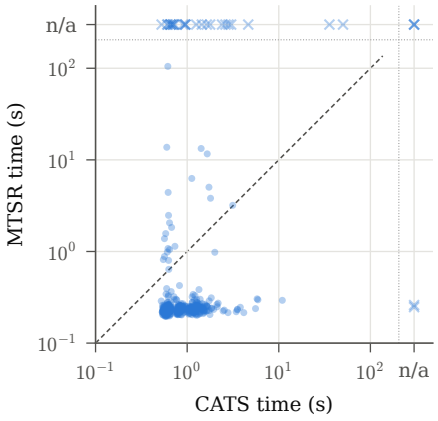


Fig. 7. *MTSR* against CATS cause checking on the 498 random candidates. Below the diagonal, *MTSR* is faster. Crosses: the tool timed-out within 120 s.

5.3.2.2 *MTSR* vs. CATS, Sketching When sketching a cause, CATS returns an LTL formula over the inputs, containing only the temporal operator ○. Since both tools return an ω-language over IP, Spot can check whether the two are equal. We use the sketching instances of CATS, with effects $○^k$ *error* (*error* is the atomic proposition). Ta-

			avg. t (s)		avg. size	
k	instances	equivalent	$MTSR$	CATS	$\|A\|$	$\|\varphi_{\mathsf{C}}\|$
1	11	11/11	0.20	1.54	2.0	1.5
2	14	14/14	0.20	1.84	2.8	4.0
3	16	16/16	0.20	2.64	3.9	8.0
4	12	9/12	0.20	3.45	5.4	12.4
5	19	17/19	0.21	3.38	6.2	20.4
6	11	9/11	0.21	2.05	7.7	26.9
7	11	7/11	0.20	9.69	9.4	35.4
8	2	0/2	0.21	50.21	12.0	46.0
all	96	83/96	0.20	4.38	5.4	15.8

Table 5. $MTSR$ against CATS sketch search, on the 96 instances both tools solve. $|A|$ is the number of states of our automaton, and $|\varphi_{\mathsf{C}}|$ is the size of CATS' formula.

ble 5 presents the results of our comparison. $MTSR$ solves all 100 instances, and CATS solves 96 of them within the timeout of 120 s.

In most cases (83 of the 96), both tools return the same language; in the other 13, $\mathcal{L}(\text{CATS}) \subsetneq \mathcal{L}(MTSR)$. This is expected, since actual causality is generally more restrictive than our definitions. The size of both answers grows with k, while the running time of $MTSR$ stays almost unchanged. This is since TraceMSR uses automata operations to find the $MTSR$, while CATS uses an explicit search to find the LTL formula.

5.3.3 *MTSR* vs. CORP [14]. CORP synthesizes a temporal actual cause as an automaton over the inputs, as we do for $MTSR$s, so the automata sizes and run-times can be compared directly. Table 7 provides a comparison over the 987 benchmarks of Table 1, out of which both tools manage to solve 902 within the given time. Generally, CORP is faster (0.25 s vs. 0.86 s on average), but generates larger automata (19.8 against 4.9 states on average). This is since sufficient reasons are typically more general, thus corresponding to smaller automata. In addition, we minimize our automata using SPOT, while CORP does not.

Table 6 provides a comparison on the instances in [14, Table 1], with the effects and the trace provided in the artifact of [14]. CORP runs with its default similarity relation $\leq^{subset}$. Its automata have the published sizes, except on full 4 with $\square\diamondsuit g_0$ (284 states here, 214 in [14]), since CORP does not minimize and its result depends on the version of spot. On every instance except one, both tools return the language of the cause φ_{C} given in [14] (found manually in [14]). On full 4 with $\square\diamondsuit g_0$, our tool exceeded its memory limit of 6 GB when trying to minimize the result; before minimization, our automaton has 46 states, against 284 for CORP. In particular, the comparison against CORP demonstrates that $MTSR$s are generally much smaller than temporal actual causes.

			t (s)		$\|A\|$		
instance	$\|T\|$	φ_{E}	CORP	$MTSR$	CORP	$MTSR$	φ_{C}
SPURIOUS 1	1	$\Diamond g_0$	0.14	0.28	1	1	*true*
SPURIOUS 2	2	$\Diamond g_0$	0.15	0.31	1	1	*true*
SPURIOUS 3	3	$\Diamond g_0$	0.11	0.30	1	1	*true*
SPURIOUS 4	4	$\Diamond g_0$	0.13	0.30	1	1	*true*
UNFAIR 2	2	$\Box\neg g_0$	0.14	0.33	1	2	$\Box r_{prio}$
UNFAIR 3	4	$\Box\neg g_0$	0.13	0.28	1	1	$\Box r_{prio}$
UNFAIR 4	6	$\Box\neg g_0$	0.12	0.28	1	1	$\Box r_{prio}$
FULL 1	1	$\Diamond g_0$	0.14	0.30	2	2	$\Diamond r_0$
		$\Box\Diamond g_0$	0.12	0.34	2	2	$\Box\Diamond r_0$
FULL 2	4	$\Diamond g_0$	0.13	0.30	2	2	$\Diamond r_0$
		$\Box\Diamond g_0$	0.14	0.29	4	2	$\Box\Diamond r_0$
FULL 3	11	$\Diamond g_0$	0.11	0.29	2	2	$\Diamond r_0$
		$\Box\Diamond g_0$	0.16	0.57	24	2	$\Box\Diamond r_0$
FULL 4	46	$\Diamond g_0$	0.15	0.30	2	2	$\Diamond r_0$
		$\Box\Diamond g_0$	11.02	MO	284	46^*	$\Box\Diamond r_0$

Table 6. $MTSR$ and CORP on the arbiters of [14, Table 1]. $|T|$ and $|A|$: number of states of the system and of the returned automaton, respectively; φ_{E} is the effect; φ_{C} is the cause given in [14]. MO: mem. out while minimizing; *size before minimization.

			avg. size		avg. t (s)	
comparison	instances	measure	ours	theirs	ours	theirs
MSR vs. [11]	152	lasso size	4.3	2.3	15.89	0.63
$MTSR$ vs. CORP	902	$\|A\|$	4.9	19.8	0.86	0.25
$MTSR$ vs. CATS sketch	96	$\|A\|$ / $\|\varphi_{\mathsf{C}}\|$	5.4	15.8	0.20	4.38

Table 7. Summary of the comparisons: Average size and running time for both tools, over the instances both answered.

6 Conclusion and Future Work

TRACEMSR is a fully automated tool for the synthesis of minimal sufficient reasons and minimal temporal sufficient reasons for reactive systems, based on the definitions of [16]. On 987 instances derived from SYNTCOMP, the $MTSR$ is computed quickly on almost all instances, and the time to compute an MSR grows mainly with the number of inputs, and less with the size of the system.

In future work, we plan to study necessary reasons, a different notion of explanation, and to apply the same definitions to systems that are not input deterministic, for which a sufficient reason does not always exist. In addition, we plan to study sufficient reasons for HyperLTL effects.

References

1. Babiak, T., Blahoudek, F., Duret-Lutz, A., Klein, J., Křetínský, J., Müller, D., Parker, D., Strejček, J.: The hanoi omega-automata format. In: Computer Aided Verification - 27th International Conference, CAV 2015, San Francisco, CA, USA, July 18-24, 2015, Proceedings, Part I. Lecture Notes in Computer Science, vol. 9206, pp. 479–486. Springer (2015). https://doi.org/10.1007/978-3-319-21690-4_31
2. Baier, C., Dubslaff, C., Funke, F., Jantsch, S., Majumdar, R., Piribauer, J., Ziemek, R.: From verification to causality-based explications (invited talk). In: Bansal, N., Merelli, E., Worrell, J. (eds.) 48th International Colloquium on Automata, Languages, and Programming, ICALP 2021, Glasgow, Scotland (Virtual Conference), July 12-16, 2021. pp. 1:1–1:20. LIPIcs, Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2021). https://doi.org/10.4230/LIPICS.ICALP.2021.1, https://doi.org/10.4230/LIPIcs.ICALP.2021.1
3. Ball, T., Naik, M., Rajamani, S.K.: From symptom to cause: localizing errors in counterexample traces. In: Aiken, A., Morrisett, G. (eds.) Conference Record of POPL 2003: The 30th SIGPLAN-SIGACT Symposium on Principles of Programming Languages, New Orleans, Louisisana, USA, January 15-17, 2003. pp. 97–105. ACM (2003). https://doi.org/10.1145/604131.604140, https://doi.org/10.1145/604131.604140
4. Barceló, P., Kozachinskiy, A., Romero, M., Subercaseaux, B., Verschae, J.: Explaining k-nearest neighbors: Abductive and counterfactual explanations. Proc. ACM Manag. Data **3**(2), 97:1–97:26 (2025). https://doi.org/10.1145/3725234, https://doi.org/10.1145/3725234
5. Beer, I., Ben-David, S., Chockler, H., Orni, A., Trefler, R.J.: Explaining counterexamples using causality. In: Bouajjani, A., Maler, O. (eds.) Computer Aided Verification, 21st International Conference, CAV 2009, Grenoble, France, June 26 - July 2, 2009. Proceedings. Lecture Notes in Computer Science, vol. 5643, pp. 94–108. Springer (2009). https://doi.org/10.1007/978-3-642-02658-4_11, https://doi.org/10.1007/978-3-642-02658-4_11
6. Beutner, R., Finkbeiner, B.: Autohyper: Explicit-state model checking for hyperltl. In: Sankaranarayanan, S., Sharygina, N. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 29th International Conference, TACAS 2023, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2023, Paris, France, April 22-27, 2023, Proceedings, Part I. Lecture Notes in Computer Science, vol. 13993, pp. 145–163. Springer (2023). https://doi.org/10.1007/978-3-031-30823-9_8, https://doi.org/10.1007/978-3-031-30823-9_8
7. Beutner, R., Finkbeiner, B., Frenkel, H., Siber, J.: Checking and sketching causes on temporal sequences. In: André, É., Sun, J. (eds.) Automated Technology for Verification and Analysis - 21st International Symposium, ATVA 2023, Singapore, October 24-27, 2023, Proceedings, Part II. Lecture Notes in Computer Science, vol. 14216, pp. 314–327. Springer (2023). https://doi.org/10.1007/978-3-031-45332-8_18, https://doi.org/10.1007/978-3-031-45332-8_18
8. Caltais, G., Guetlein, S.L., Leue, S.: Causality for general ltl-definable properties. In: Finkbeiner, B., Kleinberg, S. (eds.) Proceedings 3rd Workshop on formal reasoning about Causation, Responsibility, and Explanations in Science and Technology, CREST@ETAPS 2018, Thessaloniki, Greece, 21st April 2018. pp. 1–15. EPTCS (2018). https://doi.org/10.4204/EPTCS.286.1, https://doi.org/10.4204/EPTCS.286.1

9. Carelli, M., Finkbeiner, B., Siber, J.: Closure and complexity of temporal causality. In: 40th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2025, Singapore, June 23-26, 2025. pp. 98–111. IEEE (2025). https://doi.org/10.1109/LICS65433.2025.00015, https://doi.org/10.1109/LICS65433.2025.00015
10. Clarkson, M.R., Finkbeiner, B., Koleini, M., Micinski, K.K., Rabe, M.N., Sánchez, C.: Temporal logics for hyperproperties. In: Abadi, M., Kremer, S. (eds.) Principles of Security and Trust - Third International Conference, POST 2014, Grenoble, France, April 5-13, 2014, Proceedings. Lecture Notes in Computer Science, vol. 8414, pp. 265–284. Springer (2014), https://doi.org/10.1007/978-3-642-54792-8_15
11. Coenen, N., Dachselt, R., Finkbeiner, B., Frenkel, H., Hahn, C., Horak, T., Metzger, N., Siber, J.: Explaining hyperproperty violations. In: Shoham, S., Vizel, Y. (eds.) Computer Aided Verification - 34th International Conference, CAV 2022, Haifa, Israel, August 7-10, 2022, Proceedings, Part I. Lecture Notes in Computer Science, vol. 13371, pp. 407–429. Springer (2022). https://doi.org/10.1007/978-3-031-13185-1_20, https://doi.org/10.1007/978-3-031-13185-1_20
12. Coenen, N., Finkbeiner, B., Frenkel, H., Hahn, C., Metzger, N., Siber, J.: Temporal causality in reactive systems. In: Bouajjani, A., Holík, L., Wu, Z. (eds.) Automated Technology for Verification and Analysis - 20th International Symposium, ATVA 2022, Virtual Event, October 25-28, 2022, Proceedings. Lecture Notes in Computer Science, vol. 13505, pp. 208–224. Springer (2022). https://doi.org/10.1007/978-3-031-19992-9_13, https://doi.org/10.1007/978-3-031-19992-9_13
13. Darwiche, A., Hirth, A.: On the (complete) reasons behind decisions. J. Log. Lang. Inf. **32**(1), 63–88 (2023). https://doi.org/10.1007/S10849-022-09377-8, https://doi.org/10.1007/s10849-022-09377-8
14. Finkbeiner, B., Frenkel, H., Metzger, N., Siber, J.: Synthesis of temporal causality. In: Gurfinkel, A., Ganesh, V. (eds.) Computer Aided Verification - 36th International Conference, CAV 2024, Montreal, QC, Canada, July 24–27, 2024, Proceedings, Part III. Lecture Notes in Computer Science, vol. 14683, pp. 87–111. Springer (2024). https://doi.org/10.1007/978-3-031-65633-0_5, https://doi.org/10.1007/978-3-031-65633-0_5
15. Finkbeiner, B., Siber, J.: Counterfactuals modulo temporal logics. In: Piskac, R., Voronkov, A. (eds.) LPAR 2023: Proceedings of 24th International Conference on Logic for Programming, Artificial Intelligence and Reasoning, Manizales, Colombia, 4-9th June 2023. pp. 181–204. EPiC Series in Computing, EasyChair (2023). https://doi.org/10.29007/QTW7, https://doi.org/10.29007/qtw7
16. Frenkel, H., Rutman-Moshe, N.: Sufficient reasons and explanations for reactive systems. In: Automated Technology for Verification and Analysis - 24th International Symposium, ATVA 2026, To appear. https://arxiv.org/abs/2610.02184
17. Groce, A., Chaki, S., Kroening, D., Strichman, O.: Error explanation with distance metrics. Int. J. Softw. Tools Technol. Transf. **8**(3), 229–247 (2006). https://doi.org/10.1007/S10009-005-0202-0, https://doi.org/10.1007/s10009-005-0202-0
18. Halpern, J.Y., Pearl, J.: Causes and explanations: A structural-model approach. part i: Causes. The British Journal for the Philosophy of Science **56**(4), 843–887 (2005), http://www.jstor.org/stable/3541870
19. Hsu, T., Rabizadeh, M., Rogale, K., Filippov, F., de Oliveira Batista, M.A., Bonakdarpour, B.: HyperQB 2.0: A bounded model checker for hyperproperties. In: Darulova, E., et al. (eds.) Computer Aided Verification - 38th International

Conference, CAV 2026. Lecture Notes in Computer Science, vol. 16682, pp. 511–525. Springer (2026). https://doi.org/10.1007/978-3-032-32519-8_26
20. Hsu, T., Sánchez, C., Bonakdarpour, B.: Bounded model checking for hyperproperties. In: Groote, J.F., Larsen, K.G. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 27th International Conference, TACAS 2021, Luxembourg City, Luxembourg, March 27 - April 1, 2021, Proceedings, Part I. Lecture Notes in Computer Science, vol. 12651, pp. 94–112. Springer (2021). https://doi.org/10.1007/978-3-030-72016-2_6, https://doi.org/10.1007/978-3-030-72016-2_6
21. Ignatiev, A., Narodytska, N., Marques-Silva, J.: Abduction-based explanations for machine learning models. In: The Thirty-Third AAAI Conference on Artificial Intelligence, AAAI 2019, The Thirty-First Innovative Applications of Artificial Intelligence Conference, IAAI 2019, The Ninth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2019, Honolulu, Hawaii, USA, January 27 - February 1, 2019. pp. 1511–1519. AAAI Press (2019). https://doi.org/10.1609/AAAI.V33I01.33011511, https://doi.org/10.1609/aaai.v33i01.33011511
22. Jacobs, S., Pérez, G.A., Abraham, R., Bruyère, V., Cadilhac, M., Colange, M., Delfosse, C., van Dijk, T., Duret-Lutz, A., Faymonville, P., Finkbeiner, B., Khalimov, A., Klein, F., Luttenberger, M., Meyer, K.J., Michaud, T., Pommellet, A., Renkin, F., Schlehuber-Caissier, P., Sakr, M., Sickert, S., Staquet, G., Tamines, C., Tentrup, L., Walker, A.: The reactive synthesis competition (SYNTCOMP): 2018-2021. CoRR **abs/2206.00251** (2022). https://doi.org/10.48550/ARXIV.2206.00251
23. Leitner-Fischer, F., Leue, S.: Causality checking for complex system models. In: Giacobazzi, R., Berdine, J., Mastroeni, I. (eds.) Verification, Model Checking, and Abstract Interpretation, 14th International Conference, VMCAI 2013, Rome, Italy, January 20-22, 2013. Proceedings. pp. 248–267. Lecture Notes in Computer Science, Springer (2013). https://doi.org/10.1007/978-3-642-35873-9_16, https://doi.org/10.1007/978-3-642-35873-9_16
24. Pnueli, A.: The temporal logic of programs. In: 18th Annual Symposium on Foundations of Computer Science, Providence, Rhode Island, USA, 31 October - 1 November 1977. pp. 46–57. IEEE Computer Society (1977). https://doi.org/10.1109/SFCS.1977.32, https://doi.org/10.1109/SFCS.1977.32
25. Vardi, M.Y., Wolper, P.: An automata-theoretic approach to automatic program verification (preliminary report). In: Proceedings of the Symposium on Logic in Computer Science (LICS '86), Cambridge, Massachusetts, USA, June 16-18, 1986. pp. 332–344. IEEE Computer Society (1986)

A LTL and HyperLTL

We provide here the full definition of LTL and its semantics, and then of HyperLTL and its semantics.

LTL Linear Temporal Logic (LTL [24]) is defined over the set of atomic propositions AP using the following grammar:

$$\psi := p \mid \psi \vee \psi \mid \neg\psi \mid \bigcirc\psi \mid \psi \,\mathcal{U}\, \psi$$

Where $p \in \mathrm{AP}$, $\bigcirc$ is the temporal operator *next*, and $\mathcal{U}$ is the temporal operator *until*. We use the usual derived Boolean operators $\wedge, \rightarrow, \leftrightarrow$, and the derived temporal operators $\diamondsuit\psi = \mathit{true}\,\mathcal{U}\,\psi$ (*eventually* ψ) and $\square\psi = \neg\diamondsuit\neg\psi$ (*globally* ψ).

We define when an infinite word $\alpha \in (2^{\mathrm{AP}})^\omega$ satisfies an LTL formula φ inductively, as follows:

$$\begin{array}{ll}
\alpha \vDash p & \text{iff } p \in \alpha[0] \\
\alpha \vDash \neg\psi & \text{iff } \alpha \nvDash \psi \\
\alpha \vDash \psi_1 \vee \psi_2 & \text{iff } \alpha \vDash \psi_1 \text{ or } \alpha \vDash \psi_2 \\
\alpha \vDash \bigcirc\psi & \text{iff } \alpha[1,\infty) \vDash \psi \\
\alpha \vDash \psi_1 \,\mathcal{U}\, \psi_2 & \text{iff } \exists t_2 \in \mathbb{N} \text{ s.t. } \alpha[t_2,\infty) \vDash \psi_2 \\
 & \quad \text{and } \forall t_1 < t_2 : \alpha[t_1,\infty) \vDash \psi_1
\end{array}$$

We say that a system T satisfies an LTL formula φ, denoted by $T \vDash \varphi$ iff $\forall \alpha \in \mathrm{TR}(T) : \alpha \vDash \varphi$.

The language of an LTL formula φ is the set of all word that satisfy φ: $\mathcal{L}(\varphi) = \{\alpha \in (2^{\mathrm{AP}})^\omega \mid \alpha \vDash \varphi\}$.

It holds that every LTL formula φ can be translated to a non-deterministic Büchi automaton $\mathcal{A}$ such that $\mathcal{L}(\varphi) = \mathcal{L}(\mathcal{A})$ [25]. We denote this automaton by $\mathcal{A}_\varphi$ and will use this notation throughout the paper.

HyperLTL HyperLTL [10] extends LTL with quantification over traces, allowing it to express relational properties over infinite executions.

The syntax of HyperLTL is defined using the following grammar, given a finite set $\mathcal{V}$ of trace variables.

$$\begin{array}{l}
\psi := p_\pi \mid \psi \vee \psi \mid \neg\psi \mid \bigcirc\psi \mid \psi \,\mathcal{U}\, \psi \\
\varphi := \exists\pi.\varphi \mid \forall\pi.\varphi \mid \psi
\end{array}$$

Where $\pi \in \mathcal{V}$ is a trace variable. That is, HyperLTL formulas are LTL formulas where atomic propositions are augmented by traces variables, and, with a prefix of existential and universal trace quantifiers.

The *semantics* of HyperLTL are given with respect to a set of traces $\mathcal{T}$ and a trace assignment $\Pi : \mathcal{V} \rightarrow \mathcal{T}$. Given a trace variable $\pi \in \mathcal{V}$ and a trace $\tau \in \mathcal{T}$, we denote by $\Pi[\pi \mapsto \tau]$ the trace assignment that coincides with Π on all variables,

except for π which is mapped to τ. For the assignment Π, we denote by $\Pi[t, \infty)$ the assignment that maps every variable $\pi \in \mathcal{V}$ to the suffix of $\Pi(\pi)$ starting from position t, that is: $\Pi(\pi)[t]\Pi(\pi)[t+1]\Pi(\pi)[t+2]\cdots$. The semantics of HyperLTL is defined inductively as follows.

$$
\begin{array}{ll}
\Pi, \mathcal{T} \vDash p_\pi & \text{iff } p \in \Pi(\pi)(0) \\
\Pi, \mathcal{T} \vDash \neg\psi & \text{iff } \Pi, \mathcal{T} \nvDash \psi \\
\Pi, \mathcal{T} \vDash \psi_1 \vee \psi_2 & \text{iff } \Pi, \mathcal{T} \vDash \psi_1 \text{ or } \Pi, \mathcal{T} \vDash \psi_2 \\
\Pi, \mathcal{T} \vDash \bigcirc \psi & \text{iff } \Pi(\pi)[1, \infty), \mathcal{T} \vDash \psi \\
\Pi, \mathcal{T} \vDash \psi_1 \,\mathcal{U}\, \psi_2 & \text{iff } \exists t_2 \in \mathbb{N} \text{ s.t. } \Pi[t_2, \infty), \mathcal{T} \vDash \psi_2 \\
 & \quad \text{and } \forall t_1 < t_2 : \Pi[t_1, \infty), \mathcal{T} \vDash \psi_1 \\
\Pi, \mathcal{T} \vDash \exists \pi.\varphi & \text{iff there exists } \tau \in \mathcal{T} \text{ s.t. } \Pi[\pi \mapsto \tau], \mathcal{T} \vDash \varphi \\
\Pi, \mathcal{T} \vDash \forall \pi.\varphi & \text{iff forall } \tau \in \mathcal{T} : \Pi[\pi \mapsto \tau], \mathcal{T} \vDash \varphi
\end{array}
$$

We say that $\mathcal{T}$ satisfies ψ, denoted $\mathcal{T} \vDash \psi$, if $\Pi_\emptyset, \mathcal{T} \vDash \psi$ where $\Pi_\emptyset : \mathcal{V} \to \emptyset$ is the empty assignment, meaning that ψ contains no free trace variables that need to be assigned. We say that a transition system T satisfies a HyperLTL formula ψ if $\mathrm{TR}(T)$ satisfies ψ.

B The System of the Bounded-Search Example

Figure 8 shows the system of the example in Section 3, which shows that restricting the search to (P, L)-bounded reasons is not complete.

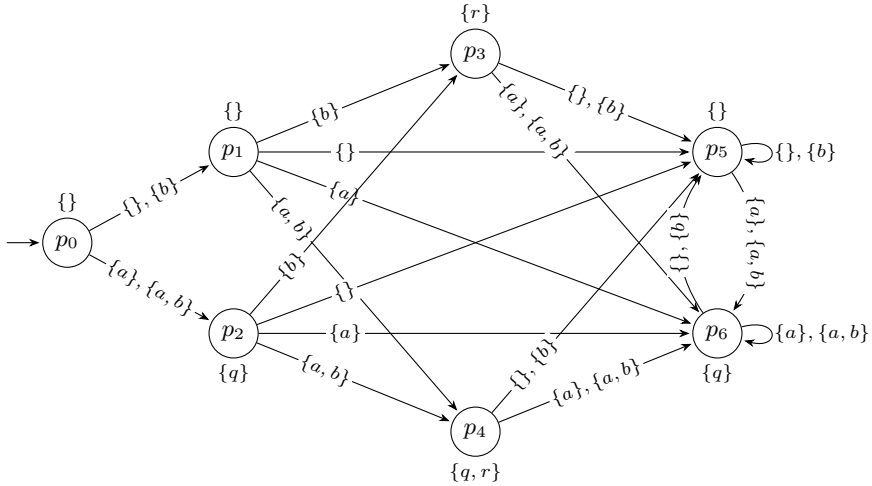


Fig. 8. The system of the bounded-search example in Section 3, over IP $= \{a, b\}$ and OP $= \{q, r\}$: the output q repeats a with a delay of one step, and r holds at time 2 iff b holds at time 1. The outputs of a step depend only on the current state and are written next to it. An edge is labeled with the sets of inputs that it reads.

C Checking a Reason with Spot

With `--hyperQB`, every reason R that HyperQB returns is checked by TraceMSR before it is reported (Section 4.2). In the following, we show that we can reduce this check to automata operations, thus limiting the effect of this step on the runtime when using HyperQB.

From the witness to a reason. HyperQB only returns the first k steps of the reason τ, so TraceMSR first translates them into a reason R. We check two options. (1) R has the shape of $\pi = u \cdot v^\omega$, i.e. a prefix of size $|u|$ and a loop of size $|v|$. We use this option if, after the first $|u|$ steps, each of the k steps is equal to the step $|v|$ positions earlier. Since k may be larger than $|u| + |v|$, the loop of π is unrolled to cover the k steps. (2) Otherwise, R is finite: it consists of the k steps, followed by $\{\}^\omega$, and the loop of π is unrolled until all k steps lie before it.

The two conditions. Since π and R are fixed, the two conditions of Φ_{MSR} (Section 4.3), with R in place of τ, become

(1) R is sufficient $\iff \forall \pi_2.\ \mathit{agree}_R(\pi, \pi_2) \rightarrow \varphi_{\pi_2}$,

$$(2)\quad R \text{ is minimal} \iff \forall \tau'.\ \tau' \subsetneq R \to \exists \pi_3.\ \mathit{agree}_{\tau'}(\pi, \pi_3) \wedge \neg \varphi_{\pi_3},$$

where π_2, π_3 range over the traces of T and τ' over all reasons. TRACEMSR checks each condition with automata.

Notation.
- $\pi = u \cdot v^\omega$ has the positions $0, \ldots, n-1$ ($n = |u| + |v|$); after $n-1$ it returns to $|u|$.
- $R[t] \subseteq \mathrm{IP}$: the inputs that R fixes at position t.
- $\ell_t(a)$: the literal a if $a \in \pi[t]$, and $\neg a$ otherwise (the value of a in π at t).
- $a_{\tau'}$: holds at time t iff the reason τ' fixes a at t (the trace representation, Section 4.3).
- $\mathcal{L}(\mathcal{A})$: the language of the automaton $\mathcal{A}$.
- $\mathcal{A} \times \mathcal{A}'$: the product of $\mathcal{A}$ and $\mathcal{A}'$, i.e. $\mathcal{L}(\mathcal{A} \times \mathcal{A}') = \mathcal{L}(\mathcal{A}) \cap \mathcal{L}(\mathcal{A}')$.
- $\mathrm{proj}_{\tau'}(\mathcal{A})$: for an automaton $\mathcal{A}$ that reads pairs (σ, τ') of a trace and a reason, $\mathrm{proj}_{\tau'}(\mathcal{A})$ reads only τ', and accepts it iff $\mathcal{A}$ accepts (σ, τ') for *some* trace σ.
- $\overline{\mathcal{A}}$: the complement of $\mathcal{A}$.

Automata for the formulas.

- $\mathcal{A}_{\neg\varphi}$: the Büchi automaton that SPOT translates from the LTL formula $\neg\varphi$.
- $\mathcal{A}_{\pi,R}$: the automaton for $\mathit{agree}_R(\pi, \sigma) = \square \bigwedge_{a \in \mathrm{IP}} \big(a_R \to (a_\pi \leftrightarrow a_\sigma)\big)$. Its states are the positions of π, and the edge that leaves t requires $\ell_t(a)$ for every $a \in R[t]$.[2]
- $\mathcal{A}_{\mathit{agree}}$: the automaton for $\mathit{agree}_{\tau'}(\pi, \sigma) = \square \bigwedge_{a \in \mathrm{IP}} \big(a_{\tau'} \to (a_\pi \leftrightarrow a_\sigma)\big)$. Since τ' is not fixed, it reads pairs (σ, τ'). It has the same states and edges as $\mathcal{A}_{\pi,R}$, and the edge that leaves t requires $a_{\tau'} \to \ell_t(a)$ for every $a \in \mathrm{IP}$.
- $\mathcal{A}_{\subsetneq R}$: the automaton for $\tau' \subsetneq R = \square \bigwedge_{a \in \mathrm{IP}} (a_{\tau'} \to a_R) \wedge \diamond \bigvee_{a \in \mathrm{IP}} (a_R \wedge \neg a_{\tau'})$. It follows the positions of R, with one bit that records whether τ' has already dropped an event of R, and accepts when the bit is set.

Sufficiency. Condition (1) fails iff some trace of T agrees with π on R and violates φ, so

$$R \text{ is sufficient} \iff \mathcal{L}(T) \cap \mathcal{L}(\mathcal{A}_{\pi,R}) \cap \mathcal{L}(\mathcal{A}_{\neg\varphi}) = \emptyset.$$

Minimality. Condition (2) quantifies over all $\tau' \subsetneq R$, which may be infinitely many. Its inner part, $\exists \pi_3.\ \mathit{agree}_{\tau'}(\pi, \pi_3) \wedge \neg \varphi_{\pi_3}$, holds exactly for the reasons τ' that are *not* sufficient, and these are accepted by

$$\mathcal{A}_{\mathit{bad}} \;=\; \mathrm{proj}_{\tau'}\big(T \times \mathcal{A}_{\mathit{agree}} \times \mathcal{A}_{\neg\varphi}\big).$$

The complement of $\mathcal{A}_{\mathit{bad}}$ accepts exactly the sufficient reasons, so

$$R \text{ is minimal} \iff \mathcal{L}(\mathcal{A}_{\subsetneq R}) \cap \mathcal{L}(\overline{\mathcal{A}_{\mathit{bad}}}) = \emptyset.$$

[2] Only $\mathcal{A}_{\neg\varphi}$ is translated from LTL. TRACEMSR builds the other three automata directly from π and R, since their formulas refer to positions of π that repeat every $|v|$ steps, which LTL cannot express in general.

$\mathcal{A}_{\subsetneq R}$ accepts all the reasons that are strictly contained in R. In particular, for an infinite R, it also accepts R with a single event dropped, which is important, as shown in the example at the beginning of Section 3. Complementation may be exponential; if $\mathcal{A}_{bad}$ has more than 400 states, TraceMSR reports R as sufficient, with minimality not decided.

D Additional Benchmark Data

Figure 9 shows the running times of MSR and [11] against the size of the system, on the 258 instances with $|\mathrm{IP}| \leq 3$ that [11] supports. [11] depends more on the system size, but stays within a few seconds.

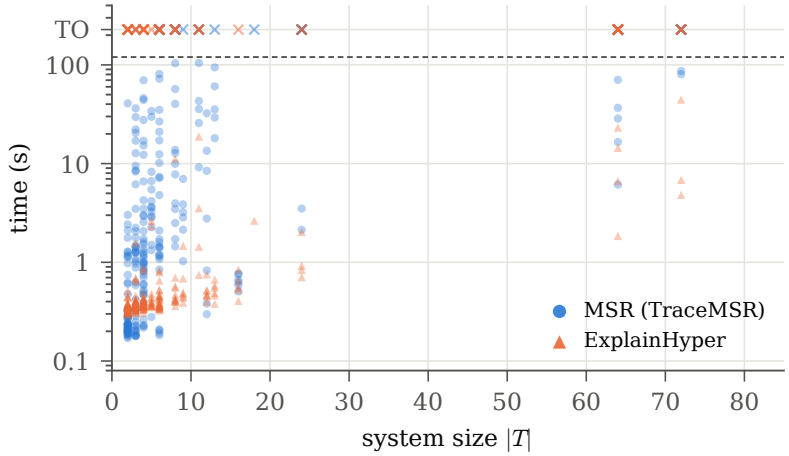


Fig. 9. MSR and [11] (ExplainHyper): running time against the number of states of the system on the 258 instances that [11] supports (one point per instance and tool; darker = more points; crosses = timeouts).

Table 8 shows whether the automaton that $MTSR$ returns can be written as a short LTL formula. We tried every formula up to size 9 (8 for $|\mathrm{IP}| = 3$, 7 for $|\mathrm{IP}| = 4$), and found one for 436 of the 913 automata (two of the 915 automata timed out when rebuilt for this analysis).

Table 2 and Figure 10 compare the two model checkers that TraceMSR uses for MSR. AutoHyper solves 703 of the 987 instances and HyperQB 381. On the 367 instances that both solve, HyperQB is faster on 240; however, AutoHyper solves 336 instances that HyperQB does not, and HyperQB only 14 that AutoHyper does not.

		found			
|IP|	automata	yes	no	avg. formula size	avg. automaton size
1	176	167	9	3.9	2.9
2	328	174	154	5.5	2.9
3	270	76	194	5.4	2.6
4	139	19	120	5.1	4.1
all	913	436	477	4.8	2.9

Table 8. The smallest LTL formula equivalent to the $MTSR$ automaton, searched up to size 9 (8 for $|\mathrm{IP}| = 3$, 7 for $|\mathrm{IP}| = 4$). 'yes' / 'no': whether one was found; the averages are over the automata for which one was found.

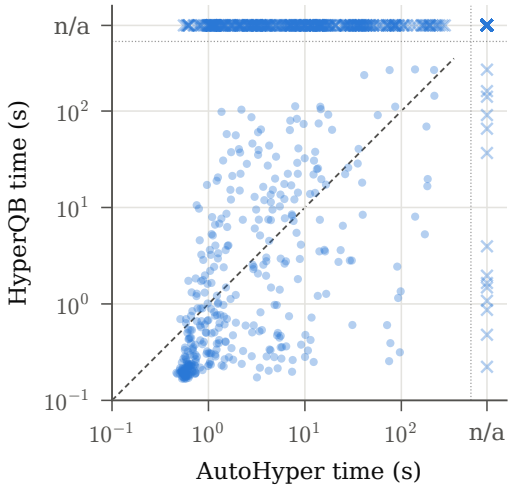


Fig. 10. MSR with AutoHyper against HyperQB on the 987 instances, one point per instance. Above the diagonal, AutoHyper is faster. Crosses (n/a): the model checker did not solve the instance within 120 s (300 s for $|\mathrm{IP}| = 4$).